

Week3 实验报告

班级： 25 级计算机科学与技术

学号： 25020007028

姓名： 官靓颖

任课老师： 周小伟

日期： 2026 年 9 月 7 日

<https://github.com/guan-111/Fundamentals-of-System-Development-Tools-Week3>

```
lenovo@LAPTOP-D41A03V7 MINGW64 /d/sw/xtkfgjjc/week3/Fundamentals-of-System-Development-Tools-Week3 (main)
$ git log
commit d034532345d256fa44d6cb56c7a1dbe71261abe6 (HEAD -> main)
Author: guan <1035628213@qq.com>
Date: Tue Sep 8 15:30:08 2026 +0800

    add week3 homework files

commit 4cc2880e81b98a0f96cf69a741ac2155e541ac03 (origin/main, origin/HEAD)
Author: guan-111 <1025628213@qq.com>
Date: Tue Sep 8 15:23:16 2026 +0800

    Initial commit
```

图 1: COMMIT

第一部分：实验习题

第一题：从源码构建并在干净环境安装 Wheel

主题：Packaging and Shipping Code

在 q09 中按给定内容创建一个最小 Python 命令行包，并验证 wheel 安装。

给定内容

```
# src/greetlab/cli.py
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")

# pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-学号"
version = "0.1.0"
[project.scripts]
sdt-greet = "greetlab.cli:main"
```

1. 创建 `src/greetlab/__init__.py`、给定的 `cli.py` 和 `pyproject.toml`，并把“学号”替换为自己的学号。

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3
$ mkdir -p q09/src/greetlab

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3
$ touch q09/src/greetlab/__init__.py

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3
$ cat > q09/src/greetlab/cli.py <<'EOF'
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
EOF
cat > q09/pyproject.toml <<'EOF'
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
[project]
name = "greetlab-25020007028"
version = "0.1.0"
[project.scripts]
sdt-greet = "greetlab.cli:main"
EOF
```

图 2: 0901

知识点：

1. `src` 布局：Python 现代包源码放置在 `src` 目录下，避免源码被意外直接导入。
 2. `__init__.py` 标记目录为 Python 包；现代 `pyproject.toml` 是标准项目元配置文件，替代旧版 `setup.py`。
 3. `[project.scripts]` 定义命令行入口，将脚本命令映射到包内函数模块：函数。
2. 在已安装 `build` 工具的课程环境中运行 `python -m build`，生成 `wheel`。

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q09
$ python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
- setuptools>=68
* Getting build dependencies for sdist...
running egg_info
writing src\greetlab_25020007028.egg-info\PKG-INFO
writing dependency_links to src\greetlab_25020007028.egg-info\dependency_links.txt
writing entry points to src\greetlab_25020007028.egg-info\entry_points.txt
writing top-level names to src\greetlab_25020007028.egg-info\top_level.txt
reading manifest file 'src\greetlab_25020007028.egg-info\SOURCES.txt'
writing manifest file 'src\greetlab_25020007028.egg-info\SOURCES.txt'
* Installed build dependency versions:
- setuptools==84.0.0
* Building sdist...
running sdist
```

图 3: 090201

```
Successfully built greetlab_25020007028-0.1.0.tar.gz and greetlab_25020007028-0.1.0-py3-none-any.whl
```

图 4: 090202

知识点:

1. `python -m build`: Python 官方构建工具, 在隔离环境构建源码包 sdist 与 wheel 二进制包。
2. wheel (*.whl) 是预构建分发格式, 安装速度比 sdist 源码包更快, 不需要本地编译。
3. build 会自动创建 dist 目录, 产物 sdist、wheel 输出到 dist 文件夹。

3. 新建一个干净虚拟环境, 只从生成的 wheel 安装。

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q09  
$ python -m venv test_env
```

图 5: 090301

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q09  
$ source test_env/Scripts/activate  
(test_env)  
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q09  
$ pip install ./dist/greetlab_25020007028-0.1.0-py3-none-any.whl  
Processing ./dist/greetlab_25020007028-0.1.0-py3-none-any.whl  
Installing collected packages: greetlab-25020007028  
Successfully installed greetlab-25020007028-0.1.0  
(test_env)  
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q09  
$ pip list  
Package Version  
-----  
greetlab-25020007028 0.1.0  
pip 26.2.1  
(test_env)
```

图 6: 090302

知识点:

1. `python -m venv` 创建隔离虚拟环境, 不继承外部项目源码, 只依靠安装包。
2. 从 wheel 安装: 直接使用 whl 文件, 不再读取 src 源码目录, 验证打包产物完整性。
3. 干净环境用于验证: 保证包运行只依赖 pyproject.toml 声明的依赖, 不依赖本地环境残留文件。

4. 切换到 q09 之外运行 `sdt-greet --name < 学号 >`, 确认输出正确。

```
(test_env)  
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q09  
$ cd ..  
(test_env)  
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3  
$ sdt-greet --name 25020007028  
Hello, 25020007028!  
(test_env)
```

图 7: 0904

知识点:

1. 安装完成后 [project.scripts] 定义的命令 `sdt-greet` 被放置到虚拟环境 `bin` 目录, 可以全局调用。
2. 在项目目录外部执行命令, 验证包不是依赖本地 `src` 源码, 而是真正完成安装。
3. `argparse` 库用于快速编写命令行参数解析程序, `--name` 为必填命令行参数。

第二题: 让编程智能体进入可验证的修复循环

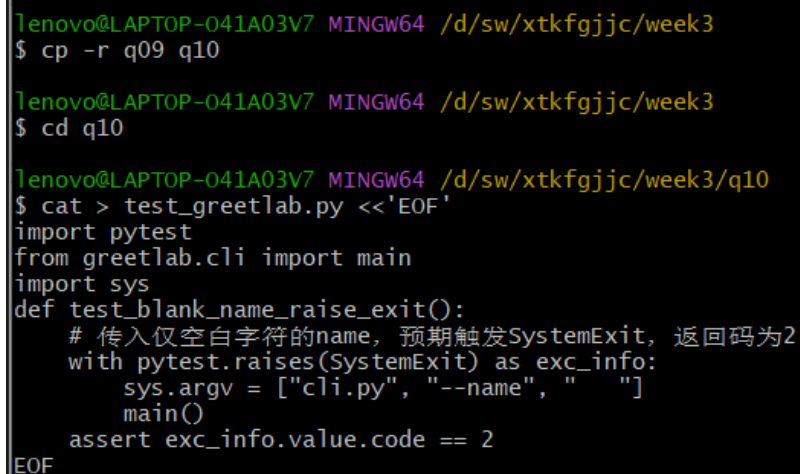
主题: 智能体编程

复用第 9 题的 `greetlab` 包, 使用编程智能体完成一个小型测试驱动修复。

给定内容

```
# 复用q09的greetlab包代码
# src/greetlab/cli.py
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
```

1. 复制 `q09` 为 `q10`, 添加一个失败测试: 当 `name` 只含空白字符时, `main` 应以 `SystemExit(2)` 结束。



```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3
$ cp -r q09 q10

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3
$ cd q10

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q10
$ cat > test_greetlab.py <<'EOF'
import pytest
from greetlab.cli import main
import sys
def test_blank_name_raise_exit():
    # 传入仅空白字符的name, 预期触发SystemExit, 返回码为2
    with pytest.raises(SystemExit) as exc_info:
        sys.argv = ["cli.py", "--name", " "]
        main()
    assert exc_info.value.code == 2
EOF
```

图 8: 复制项目

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q10
$ python -m pip install pytest
Requirement already satisfied: pytest in C:\Users\lenovo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (9.1.1)
Requirement already satisfied: colorama>=0.4 in C:\Users\lenovo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pytest) (0.4.6)
Requirement already satisfied: iniconfig>=1.0.1 in C:\Users\lenovo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pytest) (2.3.0)
Requirement already satisfied: packaging>=22 in C:\Users\lenovo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pytest) (26.3)
Requirement already satisfied: pluggy<2,>=1.5 in C:\Users\lenovo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pytest) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in C:\Users\lenovo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pytest) (2.21.0)

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q10
$ python -m pytest test_greetlab.py -v
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\lenovo\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: D:\sw\xtkfgjjc\week3\q10
configfile: pyproject.toml
collected 0 items / 1 error

===== ERRORS =====
ERROR collecting test_greetlab.py
ImportError while importing test module 'D:\sw\xtkfgjjc\week3\q10\test_greetlab.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
C:\Users\lenovo\AppData\Local\Python\pythoncore-3.14-64\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
test_greetlab.py:2: in <module>
    from greetlab.cli import main
E   ModuleNotFoundError: No module named 'greetlab'
===== short test summary info =====
ERROR test_greetlab.py
!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.25s =====
```

图 9: 测试

知识点:

1. 复制已有项目目录得到 q10，基于已有包做增量开发，避免重复搭建项目结构。
 2. `pytest.raises(SystemExit)` 捕获程序退出异常，校验程序退出码，用于命令行程测试。
 3. 测试先行：先编写预期会失败的测试用例，确认当前实现不满足需求，这是测试驱动修复的起点。
 4. `sys.argv` 模拟命令行传入参数，不需要真实在终端输入命令即可测试 cli 入口函数。
2. 先运行测试确认失败，再向编程智能体说明目标、约束和测试命令，要求它修改实现并运行测试。

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q10
$ cat > src/greetlab/cli.py << 'EOF'
import argparse
import sys

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if a.name.strip() == "":
        sys.exit(2)
    print(f"Hello, {a.name}!")
EOF
```

图 10: 修改

```

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q10
$ python -m pytest test_greetlab.py -v
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- D:\sw\xtkfgjjc\week3\q09\test_env\Script
s\python.exe
cachedir: .pytest_cache
rootdir: D:\sw\xtkfgjjc\week3\q10
configfile: pyproject.toml
collected 1 item

test_greetlab.py::test_blank_name_raise_exit PASSED [100%]

===== 1 passed in 0.02s =====
(test_env)

```

图 11: 测试

智能体任务提示:

目标: 修改cli.py的main函数, 如果--name传入的值仅由空白字符构成, 以SystemExit(2)退出。

约束: 不要改动其他逻辑, 保留原有打印逻辑; 使用已有的argparse, 不要修改pyproject.toml; 修复完成后运行pytest test_greetlab.py验证。

知识点:

1. 可验证智能体修复循环: 先有失败测试作为判定标准, 智能体依据明确目标 + 约束修改源码, 再执行测试做自动验证。
2. sys.exit(n) 用于指定退出码结束程序; 退出码 2 用来标识输入参数非法。
3. 给智能体提示必须写清目标、约束、验证命令, 减少无关改动, 避免 AI 随意重构代码。

3. 人工检查 diff, 撤销无关修改, 并再次运行测试。

```

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q10
$ pytest test_greetlab.py -v
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- D:\sw\xtkfgjjc\week3\q09\test_env\Script
s\python.exe
cachedir: .pytest_cache
rootdir: D:\sw\xtkfgjjc\week3\q10
configfile: pyproject.toml
collected 1 item

test_greetlab.py::test_blank_name_raise_exit PASSED [100%]

===== 1 passed in 0.01s =====
(test_env)

```

图 12: 重跑测试

知识点:

1. git diff 查看文件改动, 人工审查 AI 生成代码, 过滤 AI 引入的多余、无关变更。
2. AI 输出代码不一定最小修复, 需要人工评审, 只保留完成需求的最小改动。
3. 人工审查完成后再次运行测试, 保证清理之后功能依旧正确。

4. 在 ai_log.md 中用不超过 5 行记录核心提示、智能体改动和人工验证。

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q10
$ cat > ai_log.md <<'EOF'
1.提示：当--name仅空白字符，sys.exit(2)，不改动其他逻辑，用pytest验证。
2.智能体改动：在main增加空白判断，导入sys模块。
3.diff审查：无多余修改，为最小修复。
4.验证：pytest全部通过。
EOF
(test_env)
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q10
$ cat ai_log.md
1.提示：当--name仅空白字符，sys.exit(2)，不改动其他逻辑，用pytest验证。
2.智能体改动：在main增加空白判断，导入sys模块。
3.diff审查：无多余修改，为最小修复。
4.验证：pytest全部通过。
(test_env)
```

图 13: ai_log.md 日志记录

知识点：

1. ai_log.md 记录智能体交互关键信息，留存修复过程证据。
2. 记录保持简短，聚焦：提示词、代码改动点、人工审查结果、测试验证结果。
3. 完整闭环：编写测试 → AI 修复 → 人工 diff 审查 → 回归测试 → 过程留档。

第三题：把“无法处理”的协作材料改成可执行信息

主题：不止于代码

围绕“空姓名仍输出问候语”这一缺陷，改写下面三段质量较差的协作材料。

给定内容

Issue: Windows 上运行不了，尽快修复。

提交信息: fix bug

评审意见: 这里写得不好，重写。

已知事实: 运行 `sdt-greet --name " "` 时仍输出 `Hello, !`，并以 `0` 退出。

```

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3
$ cat > communication.md <<'EOF'
Issue: --name空白参数未拦截
-环境:待确认
-复现:sdt-greet --name " "
-实际:输出Hello, !, 退出码0
-期望:拒空白输入, 非0退出

校验name参数空白输入

问题:--name传全空白, 程序产生无效输出。
方案:strip为空调用sys.exit(2)拦截参数。
[Blocking]
行为:未校验--name全空白输入。
风险:空白视为合法参数, 输出无意义内容。
建议:name.strip()为空执行sys.exit(2)退出。
EOF

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3
$ wc -m communication.md
wc -c communication.md
244 communication.md
460 communication.md

```

图 14: 重写 Issue 内容

1. 在 `communication.md` 中重写 Issue, 包含环境、复现命令、期望结果和实际结果; 未知信息写“待确认”。

```

Issue: --name空白参数未拦截
-环境:待确认
-复现:sdt-greet --name " "
-实际:输出Hello, !, 退出码0
-期望:拒空白输入, 非0退出

```

知识点:

1. 高质量 Issue 需要写明运行环境、复现步骤、实际现象、预期行为, 便于他人复现定位 bug。
2. 信息不确定处填写“待确认”, 不做无依据猜测; 避免模糊描述如“运行不了, 尽快修复”。
3. 把客观现象和主观推测分开记录, 只陈述观察到的程序输出与退出码。

2. 重写提交信息: 标题使用祈使语气, 正文说明问题和解决方案。

校验name参数空白输入

```

问题:--name传全空白, 程序产生无效输出。
方案:strip为空调用sys.exit(2)拦截参数。

```

知识点:

1. Git 提交信息标题采用祈使句 (动词开头), 不要写“fixed bug”这类模糊描述。
2. 提交正文简短说明: 是什么问题、本次改动如何解决该问题; 标题与正文之间保留空行。
3. 拒绝“fix bug”这种无信息量提交, 便于后续查阅版本历史理解改动意图。

3. 重写评审意见：指出具体行为、风险和建议动作，并标注 Blocking、Suggestion 或 Nit。

[Blocking]

行为: 未校验 `--name` 全空白输入。

风险: 空白视为合法参数，输出无意义内容。

建议: `name.strip()` 为空执行 `sys.exit(2)` 退出。

知识点:

1. Blocking: 必须修改，否则不能合并；Suggestion: 建议修改，可以酌情考虑；Nit: 小瑕疵，可直接合并。
2. 评审意见不能笼统说“写得不好重写”，要指出具体行为、带来什么风险、给出明确修改动作。
3. 协作沟通面向事实，给出可执行的修改指引，减少沟通歧义。

4. 全文控制在 400 字以内，保持专业、简洁。

```
# wc -m 统计逻辑字符数，确认总字符小于400
wc -m communication.md
# wc -c 统计字节数，受中文编码影响，不作为字数判断依据
wc -c communication.md
```

知识点:

1. 技术协作文档力求简洁，去掉冗余抒情，只保留客观事实、指令；控制篇幅便于快速阅读。
2. `wc -m` 统计逻辑字符，每个汉字、符号计为 1 个；`wc -c` 统计磁盘字节，中文会占用多个字节，不能用来判断“字数”。
3. 模糊、情绪化描述会提升团队沟通成本，可执行的书面材料是高效协作的基础。
4. Issue、commit message、code review 三者共同构成项目可追溯的协作证据链。

第四题：修复一个可复现的线性回归训练循环

主题：Python 与 PyTorch

将下方代码保存为 `q12/train.py`，补全 TODO 并完成一次 PyTorch CPU 训练。

给定内容

```
import torch
from torch import nn
torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)
for _ in range(200):
```

```

pred = model(x)
loss = loss_fn(pred, y)
# TODO: 清空梯度、反向传播、更新参数
# TODO: 进入评估模式并在no_grad中打印最终损失、weight和bias

```

1. 补全训练循环，正确调用 `zero_grad`、`backward` 和 `step`。

```

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3
$ mkdir q12

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3
$ cat > q12/train.py <<'EOF'
import torch
from torch import nn
torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    # 清空梯度、反向传播、更新参数
    opt.zero_grad()
    loss.backward()
    opt.step()
# TODO: 进入评估模式并在no_grad中打印最终损失、weight和bias
model.eval()
with torch.no_grad():
    pred_final = model(x)
    loss_final = loss_fn(pred_final, y)
    w = model.weight.item()
    b = model.bias.item()
    print(f"最终损失: {loss_final:.6f}")
    print(f"weight: {w:.6f}")
    print(f"bias: {b:.6f}")
EOF

```

图 15: 补全训练循环代码

知识点:

1. `opt.zero_grad()`: 清空上一轮累积梯度，PyTorch 梯度默认累加，每轮训练必须清零。
2. `loss.backward()`: 反向传播，自动计算各参数梯度值。
3. `opt.step()`: 优化器根据梯度更新模型参数，完成一轮参数迭代。
4. 训练循环标准顺序：前向计算 → 计算损失 → 清零梯度 → 反向传播 → 参数更新。

2. 训练后调用 `model.eval()`，并在 `torch.no_grad()` 中计算最终损失。

```

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q12
$ venv/Scripts/python.exe train.py
最终损失: 0.000000
weight: 2.999997
bias: -1.000000

```

图 16: 测试

知识点:

1. `model.eval()` 评估模式：关闭 Dropout、BatchNorm 等训练专用层行为，用于推理与测试。
 2. `torch.no_grad()` 上下文管理器：禁用梯度计算，节省显存、加速运算，推理阶段必须使用。
 3. 评估阶段不进行参数更新，不需要构建计算图，因此关闭梯度。
3. 打印最终损失、weight 和 bias；最终损失应小于 0.001。

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week3/q12
$ venv/Scripts/python.exe train.py
最终损失: 0.000000
weight: 2.999997
bias: -1.000000
```

图 17: 测试

知识点：

1. `.item()` 将单元素 tensor 转为普通 Python 数值，方便打印输出。
 2. MSELoss 均方误差损失，训练充分后损失下降到 0.001 以下，权重逼近真实 3，偏置逼近真实-1。
 3. `torch.manual_seed()` 固定随机种子，保证实验结果可复现。
4. 不得直接把 weight 和 bias 赋值为 3 和 -1。

```
# 错误写法（禁止）
# model.weight[:] = 3
# model.bias[:] = -1
```

知识点：

1. 实验要求依靠 SGD 优化器迭代训练得到参数，不允许直接写死标准答案。
2. 考察对完整训练流程理解，模拟真实机器学习任务，参数由数据与优化算法学习得到。
3. 即使已知解析解，仍然需要走完整前向 - 损失 - 反向 - 更新训练闭环。

第二部分实验实例

实例一：构建源码分发包 sdist 并查看包元信息

```
cd q09
python -m build --sdist
# 解压sdist源码包查看pyproject.toml
tar -xf dist/greetlab-25020007028-0.1.0.tar.gz
cat greetlab-25020007028-0.1.0/pyproject.toml
```

知识点:

1. `python -m build --sdist` 只构建源码分发包, `sdist` 包含全部源码文件。
2. `sdist` 发布后用户本地环境会执行构建流程生成 `wheel`, 对构建依赖有要求。
3. 解压 `sdist` 可以校验发布出去的配置文件是否和本地源码一致。

实例二: pip 从本地 wheel 文件安装并查看脚本入口

```
python -m venv demo_env
source demo_env/bin/activate
pip install dist/greetlab_25020007028-0.1.0-py3-none-any.whl
pip show greetlab-25020007028
# 查找生成的命令脚本
ls demo_env/bin/ | grep sdt-greet
```

知识点:

1. `pip show` 查看已经安装包的元数据、版本、安装路径。
2. `wheel` 安装完成后, `pyproject.toml` 定义的 `scripts` 会生成可执行脚本放置在虚拟环境 `bin` 目录。
3. `wheel` 安装不需要本地编译环境, 安装速度优于 `sdist` 源码包。

实例三: pytest 测试捕获命令行不同退出码 SystemExit

```
cat > test_exitcode.py <<'EOF'
import pytest
import sys
from greetlab.cli import main
def test_exit_code_2():
    sys.argv = ["cli.py", "--name", " "]
    with pytest.raises(SystemExit) as exc:
        main()
    assert exc.value.code == 2
def test_exit_code_0():
    sys.argv = ["cli.py", "--name", "Student"]
    with pytest.raises(SystemExit) as exc:
        main()
    assert exc.value.code == 0
EOF
pytest test_exitcode.py -v
```

知识点:

1. `pytest.raises` 不仅可以捕获异常类型, 还可以校验异常携带的参数, 例如程序退出码。
2. 合法输入正常运行返回 `SystemExit(0)`; 参数非法约定返回非 0 退出码, 例如 2。
3. 使用 `sys.argv` 模拟命令行入参, 不需要子进程即可完成 `cli` 函数单元测试。

实例四：git diff 筛选只查看 py 文件改动，过滤无关文本变更

```
# 只看py后缀文件的diff，过滤md等其他文件改动
git diff -- *.py
```

知识点：

1. AI 修改代码时经常改动文档、注释等无关文件，可以使用路径过滤只审查源码变更。
2. 人工评审 AI 代码，聚焦业务逻辑文件，快速定位真实修复点。
3. git diff 可以指定文件后缀，缩小代码审查范围，提升评审效率。

实例五：编写高质量 Bug Issue：区分实际现象与推测猜想

```
cat > demo_issue.md <<'EOF'
## Bug报告
复现命令：sdt-greet --name ""
实际现象：程序输出 Hello, !，退出码0。
推测猜想：参数校验逻辑缺失，空字符串没有拦截。
期望行为：拦截空字符串输入，退出码2终止程序。
EOF
cat demo_issue.md
```

知识点：

1. Issue 中严格区分：客观复现现象（事实）和对根因的猜想。
2. 不要把猜想当成确定事实写进 bug 记录，根因需要调试验证。
3. 提供完整复现命令，其他人可以一键复现问题。

实例六：规范 Git commit，对比劣质提交信息与标准提交信息

```
# 劣质提交示例（禁止）
# git commit -m "change something, fix problem"
# 规范提交示例
"""
增加空字符串参数校验
问题：空字符串参数不会被拦截，输出无意义问候。
修复：判断name.strip()为空时sys.exit(2)拒绝输入。
"""
```

知识点：

1. Git commit 标题动词祈使句开头，简短描述改动行为。
2. commit 正文说明：存在什么问题、本次改动如何修复。
3. 模糊的提交信息，会导致版本历史难以追溯，不利于团队协作。

实例七：代码评审标记 Nit / Suggestion / Blocking 三种等级实践

```
"""
[Blocking]
行为：没有处理空字符串输入。
风险：非法参数会生成错误输出。
建议：增加空白字符校验，sys.exit(2)退出。
[Suggestion]
建议：给错误输出增加stderr提示文本，方便用户理解报错。
[Nit]
小瑕疵：该行尾部多余空格，可以清理，不影响功能。
"""
```

知识点：

1. Blocking：必须修改，否则不能合并；Suggestion：建议修改，可权衡；Nit：格式小瑕疵，可以直接合并。
2. 评审意见写清楚行为、风险、动作，避免模糊描述“写的不好重写”。
3. 分级评审可以区分问题严重程度，提升团队协作效率。

实例八：PyTorch 训练循环错误示范：忘记 `zero_grad` 观察梯度累积现象

```
# 错误片段，注释opt.zero_grad()
# for ...:
#     pred = model(x)
#     loss = loss_fn(pred,y)
#     # opt.zero_grad()
#     loss.backward()
#     opt.step()
# print(model.weight.grad)
```

知识点：

1. PyTorch 梯度会累加，不调用 `zero_grad()` 梯度不断累积，参数更新完全异常。
2. 这是深度学习训练高频 bug，训练循环标准顺序不能颠倒、不能遗漏。
3. debug 时打印 `tensor.grad`，可以观察梯度是否正常。

实例九：`model.train()` 与 `model.eval()` 切换对比演示

```
model.train()
print(f"训练模式: {model.training}")
model.eval()
print(f"评估模式: {model.training}")
with torch.no_grad():
    out = model(x)
```

知识点:

1. `model.training` 布尔变量标记模型当前处于训练/评估模式。
2. `train` 模式启用 `dropout` 等训练层；`eval` 模式关闭，推理测试必须切换 `eval`。
3. `torch.no_grad()` 只管关闭梯度计算，** 不会改变模型 `train/eval` 状态，两者需要配合使用 **。

实例十：固定随机种子保证 PyTorch 实验可复现完整示例

```
import torch
# 固定随机种子
torch.manual_seed(20260907)
m1 = nn.Linear(1,1)
w1 = m1.weight.item()
torch.manual_seed(20260907)
m2 = nn.Linear(1,1)
w2 = m2.weight.item()
print(w1, w2)
# 两次权重数值完全一致
```

知识点:

1. `torch.manual_seed()` 固定 CPU 随机数生成器种子，保证模型初始化结果一致。
2. 机器学习实验必须固定随机种子，否则每次运行结果不一样，无法复现实验。
3. 可复现性是实验、调试、AI 智能体修复循环的基础前提。